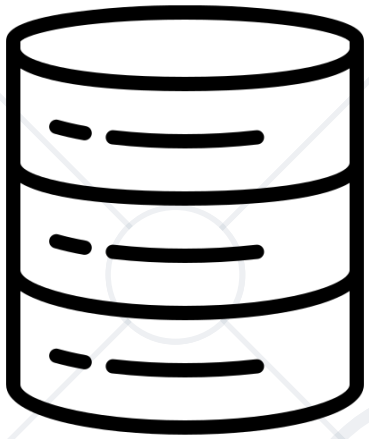
PostgreSQL Introduction. Working with Tables.



SoftUni Team

Technical Trainers

Software University

SoftUni

Software University

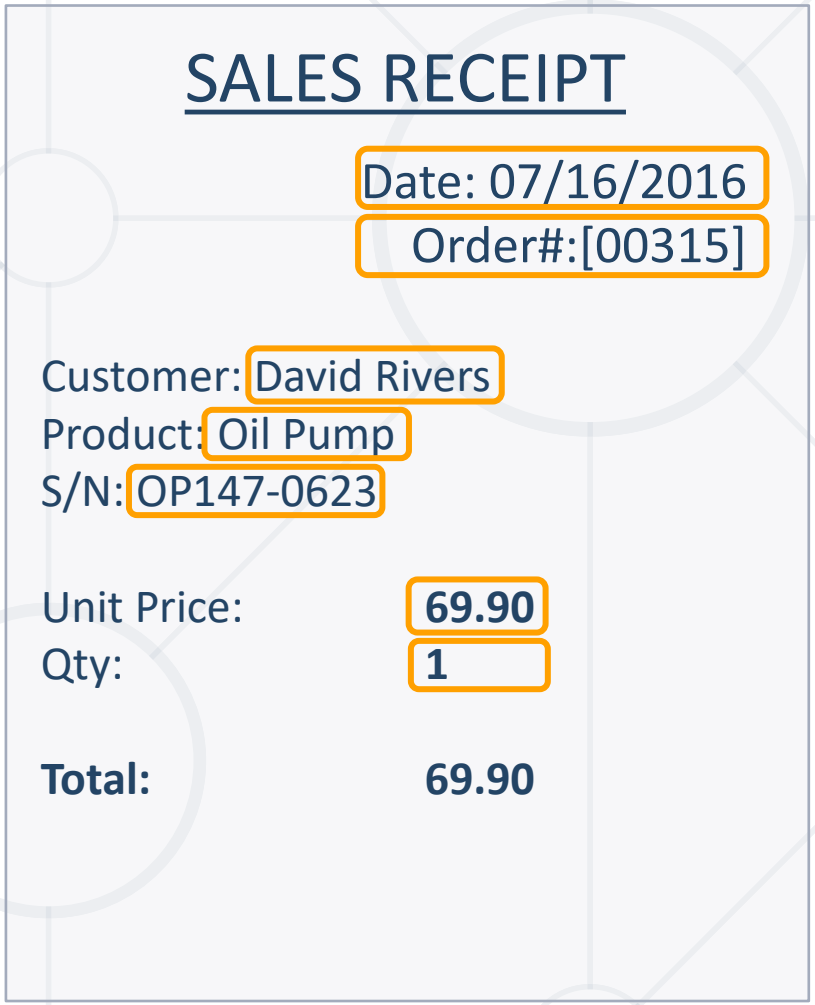
https://softuni.bg

Table of Contents

Data Management

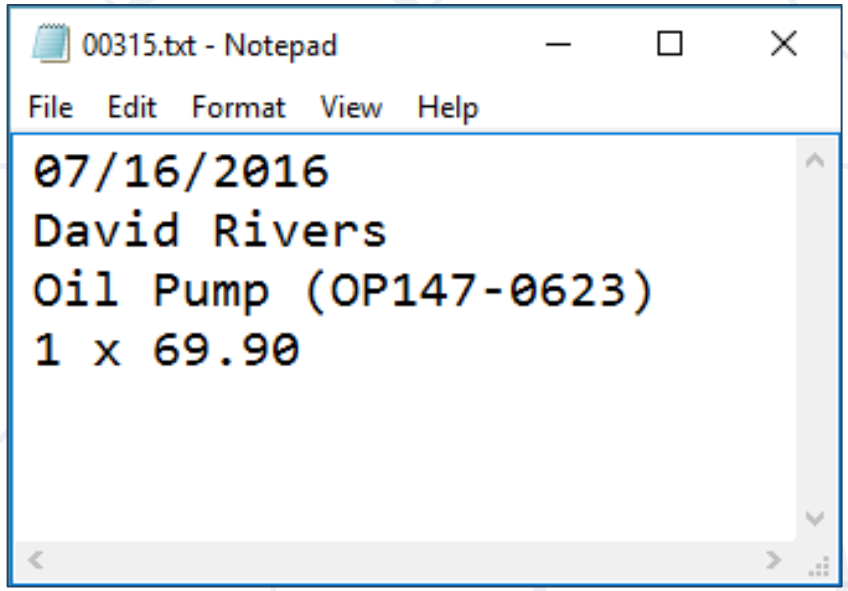
When Do We Need a Database?

Storage vs. Management



00315 – 07/16/2016
David Rivers
Oil Pump (OP147-0623)
1 x 69.90

Storage vs. Management



Order#	Date	Customer	Product	S/N	Qty
00315	07/16/2016	David Rivers	Oil Pump	OP147-063	1

Database

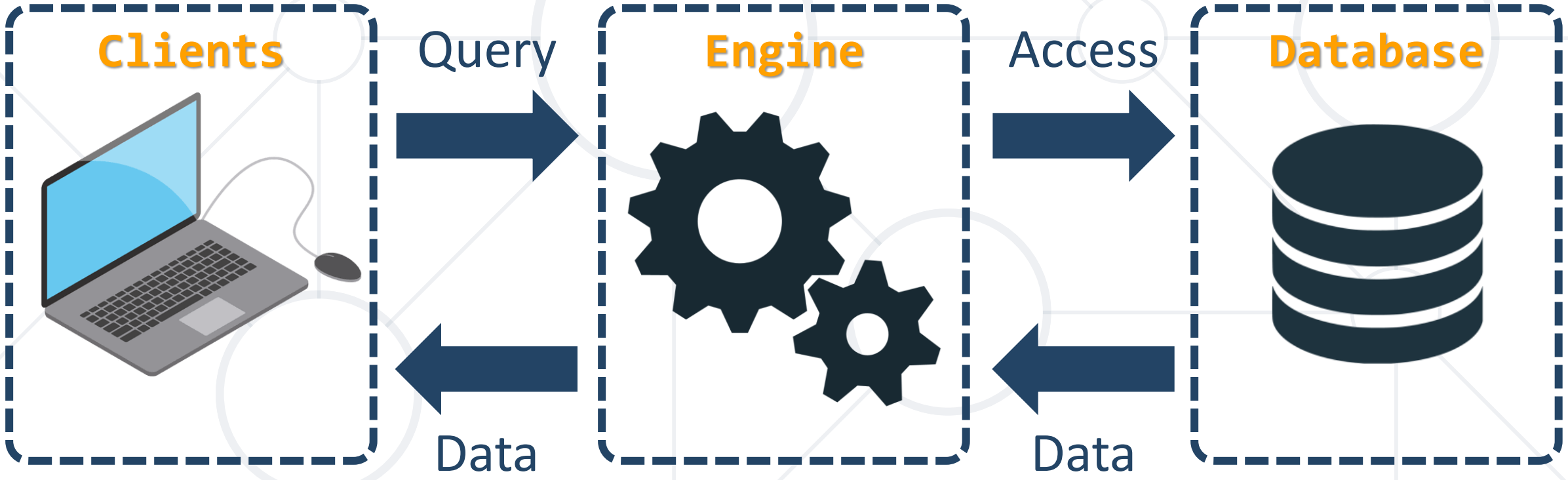
- A database is an **organized** collection of **related** information
  - The user doesn't have **direct access** to the stored data
  - Access to data is usually **provided by a DBMS**

Database Management System

- DataBase Management System
  - **Provides tools** to define, manipulate, retrieve and manage data in a database
  - **Parses requests** from the user and takes the appropriate action

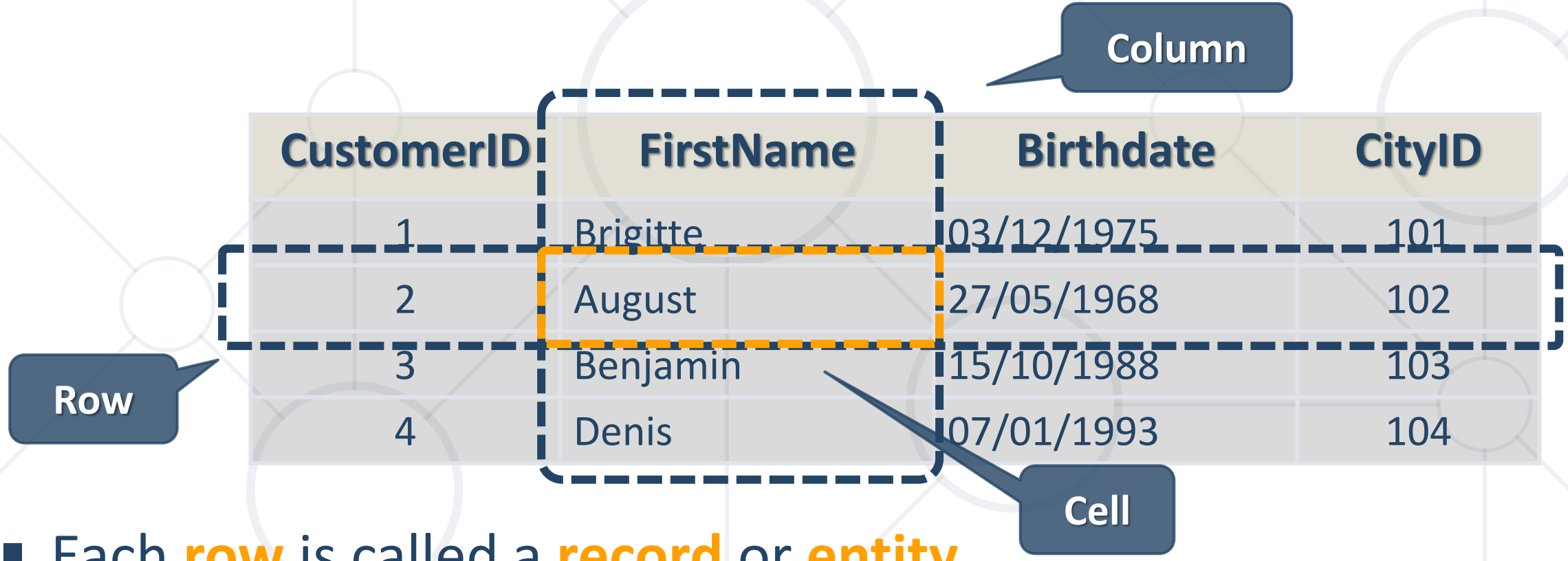
Database Engine Flow

- PostgreSQL uses the Client-Server Model

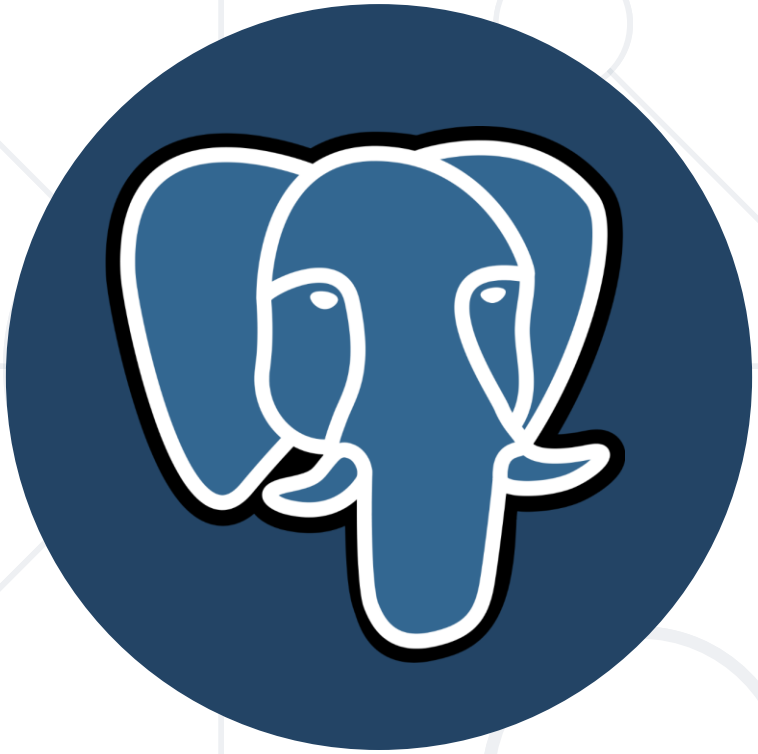


Database Table Elements

- The table is the main **building block** of any database



- Each **row** is called a **record** or **entity**
- Columns (**fields**) define the **type** of data they contain



PostgreSQL

What is PostgreSQL?

- Object–Relational Database Management System (ORDBMS)
- Widely used open-source cross-platform system

Rank			DBMS	Database Model
Jan 2023	Dec 2022	Jan 2022		
1.	1.	1.	Oracle	Relational, Multi-model
2.	2.	2.	MySQL	Relational, Multi-model
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model
4.	4.	4.	PostgreSQL	Relational, Multi-model
5.	5.	5.	MongoDB	Document, Multi-model
6.	6.	6.	Redis	Key-value, Multi-model
7.	7.	7.	IBM Db2	Relational, Multi-model
8.	8.	8.	Elasticsearch	Search engine, Multi-model
9.	9.	9.	Microsoft Access	Relational
10.	10.	10.	SQLite	Relational

What Makes PostgreSQL Stand Out?

- First DBMS that implements **multi-version concurrency control feature**
  - Data can be safely read and updated at the same time
- Able to add **custom functions**
- Designed to be **extensible**
- Defining custom data types, plugins, etc.
- Very active community

Structured Query Language

What is SQL?

- **Programming** language
- Designed for **managing** data in a **relational** database
  - Access **many records** with **one single command**
  - **Eliminates** the need to specify **how** to reach a record
- Developed at **IBM** in the early 1970s

SQL Queries

- We communicate with the database engine using SQL
- Queries provide greater **control** and **flexibility**
- To create a database using SQL:

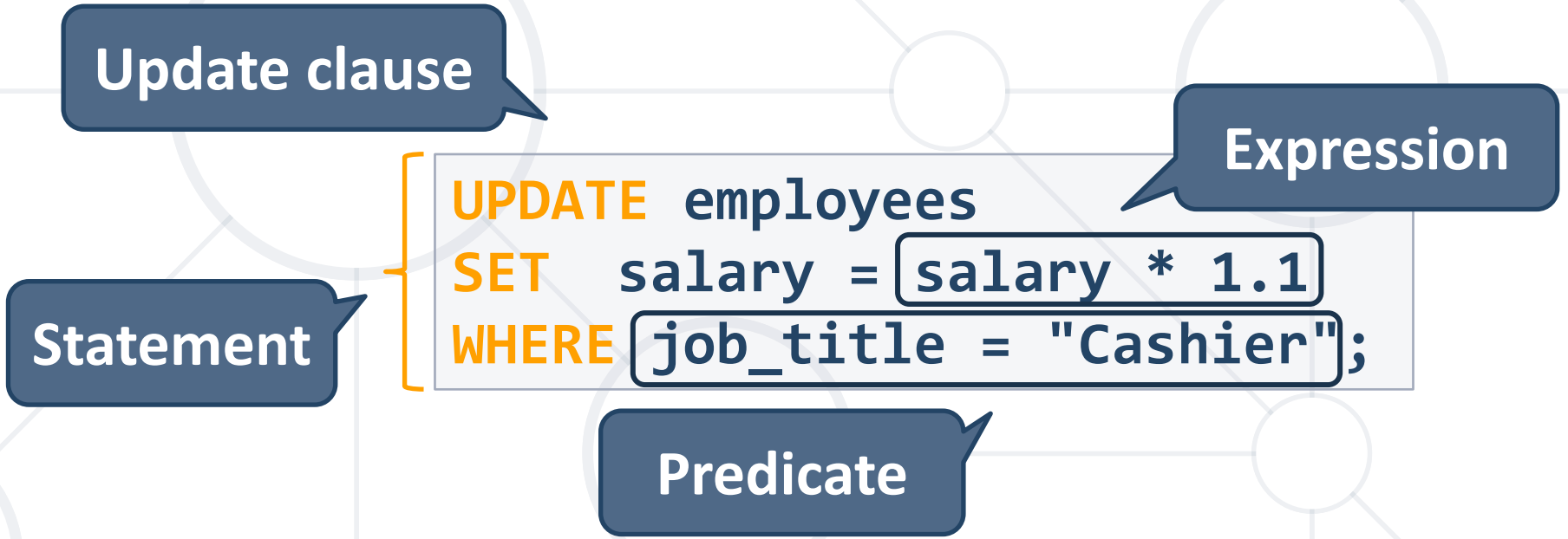
Database name

```
CREATE DATABASE MyFirstDB;
```

- SQL keywords are conventionally **capitalized**

SQL Elements

- Subdivided into several language elements
  - Queries
  - Clauses
  - Expressions
  - Predicates
  - Statements

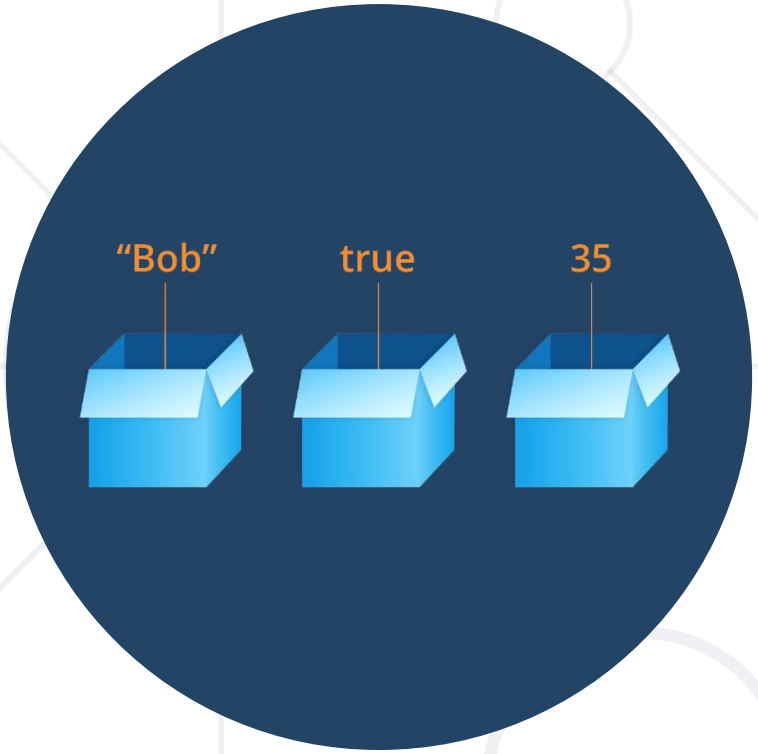


SQL vs NoSQL Databases

- **SQL Database**:
  - **Relational** database management system
  - **Predefined** Schema
  - Suited for **complex** queries
  - **Vertically** scalable
- **NoSQL database**:
  - **Non-relational** database system
  - **Dynamic** Schema
  - Suited for **hierarchical** data storage
  - **Horizontally** scalable

Live Demo

Create a New Database in DBeaver



Data Types in SQL

Numeric Data Types

- **Integer types**
  - SMALLINT, INTEGER/INT, BIGINT
- **Arbitrary Precision Numbers**
  - DECIMAL, NUMERIC
- **Floating-Point Types**
  - REAL, DOUBLE PRECISION
- **Serial Types**
  - SMALLSERIAL, SERIAL, BIGSERIAL

The type INTEGER/INT is the common choice

Recommended for storing quantities where exactness is required

Storing and retrieving a value might show a slight difference

Used for creating unique identifier columns

String Data Types

- **CHARACTER/CHAR**[(M)]
  - Fixed-length e.g., CHAR(30)
  - CHAR without the length specifier (m) is the same as CHAR(1)
- **CHARACTER VARYING/VARCHAR**[(N)]
  - Variable-length with limit e.g., VARCHAR(30)
  - VARCHAR without (n) can store a string with unlimited length
- **TEXT**
  - Stores strings of any length

CHAR vs VARCHAR

- Storing data in CHAR and VARCHAR examples

Value	CHAR(4)	Storage Required	VARCHAR(4)	Storage Required
''	' '	4 bytes	''	1 bytes
'ab'	'ab '	4 bytes	'ab'	3 bytes
'abcd'	'abcd'	4 bytes	'abcd'	5 bytes
'abcdefgh'	'abcd'	4 bytes	'abcd'	5 bytes

Date Types

- **DATE** - for values with a date part but **no time part**
  - 2016-06-23
- **TIME** - for values with time but **no date part**
  - 14:01:10
- **TIMESTAMP** - both date **and** time parts
  - 2020-10-05 14:01:10
- **TIMESTAMPTZ** - both date **and** time parts **with time zone**
  - 2020-10-05 14:01:10+02:00

Lexical Structure in PostgreSQL

Lexical Structure in PostgreSQL

- Keywords and unquoted table/column names are **case insensitive**

```
CREATE TABLE CUSTOMER();
create table customer();
CrEaTe tAbLe CuStOmEr();
```

These are equivalent

- The convention is to write the **keywords in uppercase** and the **unquoted names in lowercase**

```
CREATE TABLE customer();
```

Lexical Structure in PostgreSQL

- **String literals** are enclosed in ['](single quotes)
  - String separated by one or more **newlines** can be **concatenated as one string**

```
'Bartho'
'lomew'
```

```
'Bartholomew'
```

- **Table** and **column** names containing **special symbols** use ["] (double quotes)
  - Allows constructing names that would otherwise not be possible e.g., `"select"`, `"Full Name"`
  - Makes them **case-sensitive**

Comments in PostgreSQL

- Adding a comment on a **single line**

```
-- comment
```

```
CREATE DATASABE db_name; -- comment
```

- Adding a comment on **multiple lines** or anywhere **in the SQL statement**

```
CREATE TABLE /* comment */ ();
```

Live Demo

Create a New Table

Retrieving Data

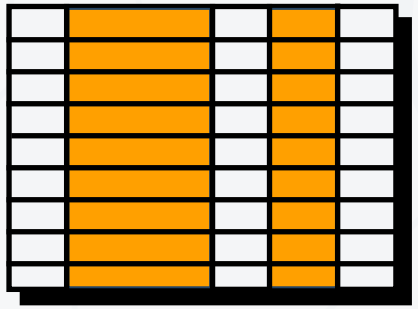
Using SQL SELECT

Capabilities of SQL SELECT

Software University

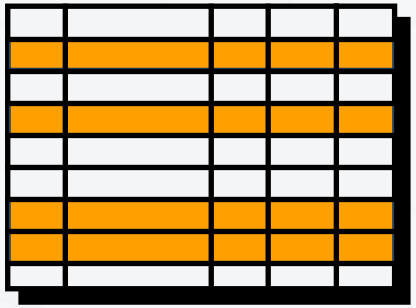
Projection

Take a subset of the columns



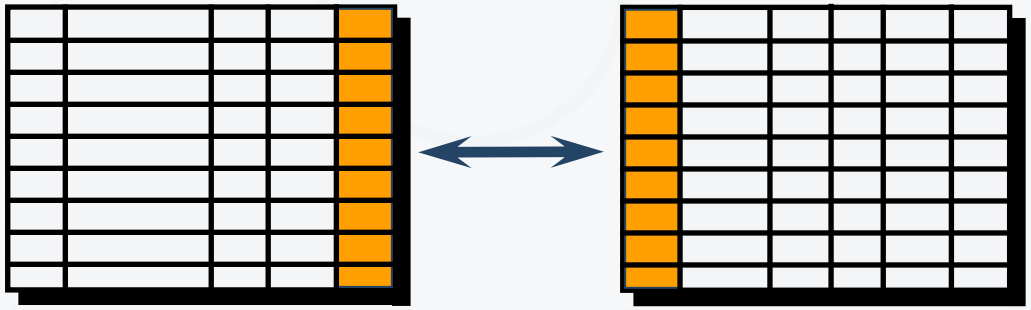
Selection

Take a subset of the rows



Join

Combine tables by some column



Retrieve / Read Records Using SQL

- Get **all** information from a table

id	first_name	last_name	room_id
1	Peter	Petrov	1
2	George	Georgiev	2
3	Mariya	Marieva	28
…	…	…	…

Retrieve / Read Records Using SQL

- Get **specific** columns from a table

List of columns

```
SELECT first_name, last_name FROM clients;
```

first_name	last_name
Peter	Petrov
George	Georgiev
Mariya	Marieva
…	…

Column Aliases

- Aliases **rename** a table or a column **heading**

Display Name

```
SELECT p.first_name AS "First Name"
FROM clients AS p;
```

First Name
Peter
George
…

- Hint: use it to **shorten fields** or **clarify abbreviations**

Concatenation Operator

- You can **concatenate** column records using the || operator

```
SELECT first_name || ' ' || last_name AS "Full Name"
FROM clients;
```

Full Name
Peter Petrov
George Georgiev
…

Sorting with ORDER BY

- Sort rows with the **ORDER BY** clause
  - **ASC**: ascending order, default
  - **DESC**: descending order

```
SELECT last_name, salary
FROM employees
ORDER BY salary;
```

last_name	salary
Johnson	880
Smith	900
Petrov	990
…	…

```
SELECT last_name, salary
FROM employees
ORDER BY salary DESC;
```

last_name	salary
Petrov	2100
Jackson	1800
Ivanov	1600
…	…

Filtering Selected Rows

Using SQL WHERE Clause

Filtering the Selected Rows

- Eliminate duplicate results

```
SELECT DISTINCT first_name FROM employees;
```

- Eliminate duplicate results based on the combination of values

```
SELECT DISTINCT first_name, last_name FROM employees;
```

- Eliminate duplicate results based on the first column

```
SELECT DISTINCT ON (first_name) first_name, last_name
FROM employees;
```

Filtering the Selected Rows

- Filter rows by specific conditions

Clause

```
SELECT id, first_name, last_name
FROM employees
WHERE id <= 2;
```

Comparison Operator

id	first_name	last_name
1	John	Smith
2	John	Johnson

Comparison Operators

Description	Operator
Equal to	=
Different from	!=, <>
Greater than	>
Greater than or equal to	>=
Less than	<
Less than or equal to	<=

Logical Operators

- To allow the existence of multiple conditions

```
SELECT last_name FROM employees
WHERE salary = 900 AND first_name = 'John';
```

- To reverse the meaning of the logical operator

```
SELECT last_name FROM employees
WHERE NOT salary = 900;
```

- To combine multiple conditions

```
SELECT last_name FROM employees
WHERE salary = 900 OR salary = 1100;
```

Other Comparison Conditions

- Use **IN** to specify a set of values:

```
SELECT first_name, last_name
FROM employees
WHERE salary IN (2100, 1100, 900, 880);
```

- Use **NOT IN** to specify a set of values:

```
SELECT first_name, last_name
FROM employees
WHERE salary NOT IN (2100, 1100, 900, 880);
```

Data Manipulation

Using SQL INSERT, UPDATE, DELETE

Inserting Data

- Insert a record in a table

```
INSERT INTO towns VALUES (33, 'Paris');
```

id	name
33	Paris

- Insert data into specific columns of a table

```
INSERT INTO towns(name) VALUES ('Sofia');
```

id	name
33	Paris
34	Sofia

- **Bulk data** can be recorded in a single query

```
INSERT INTO towns(name)
     VALUES ('London'),
            ('Rome');
```

id	name
33	Paris
34	Sofia
35	London
36	Rome

Inserting Data

- You can use existing records to create a **new table**

```
CREATE TABLE customer_data
AS SELECT id, last_name, room_id
FROM clients;
```

New table name

Existing source

- Or into an existing table

List of columns

```
INSERT INTO projects(name, start_date)
SELECT name || ' Restructuring', '2023-01-01'
FROM departments;
```

Updating Data

- The SQL **UPDATE** command

New values

```
UPDATE employees
   SET last_name = 'Brown'
 WHERE id = 1;
```

Multiple updates

```
UPDATE employees
   SET salary = salary * 1.10,
       job_title = 'Senior ' || job_title
 WHERE department_id = 3;
```

- Note: If you **skip the WHERE** clause, **all rows** in the table will be updated

Deleting Data

- Deleting **specific rows** from a table

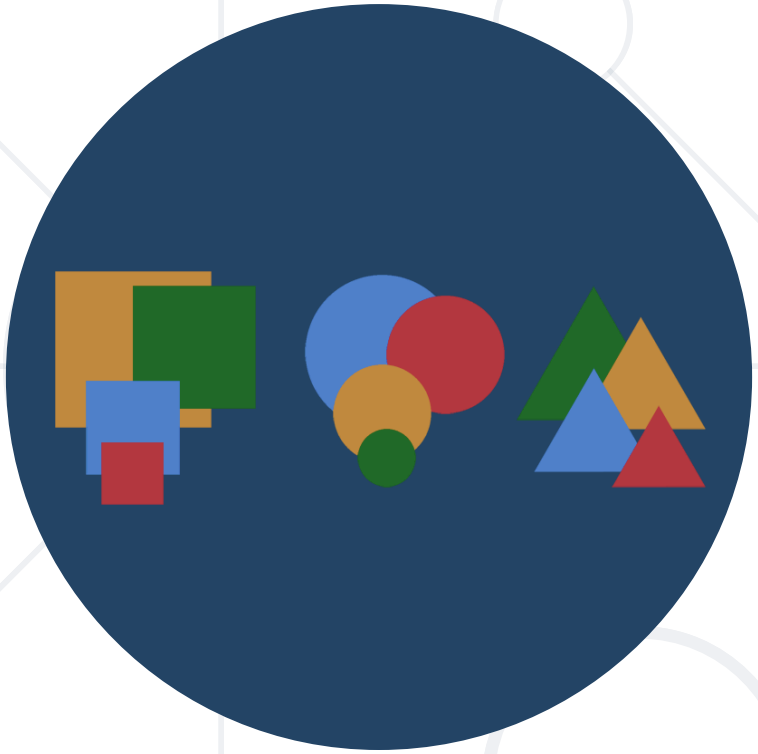
```
DELETE FROM employees
WHERE id = 1;
```

Condition

  - Note: If you **skip the WHERE** clause, **all rows** in the table will be deleted

- Delete all rows from a table
  - **TRUNCATE** works faster than **DELETE**

```
TRUNCATE TABLE employees;
```



Grouping

Consolidating Data Based On Criteria

Grouping

- Grouping allows taking data into **separate groups** based on a **common property**

Grouping column

employee	department_name	salary
Adam	Database Support	5,000
John	Database Support	15,000
Jane	Application Support	10,000
George	Application Support	15,000
Lila	Application Support	5,000
Fred	Software Support	15,000

Can be aggregated

Example: Grouping

- Get the total salaries of all employees **grouped by department**

employee	department_name	salary
Adam	Database Support	5,000
John	Database Support	15,000
Jane	Application Support	10,000
George	Application Support	15,000
Lia	Application Support	5,000
Fred	Software Support	15,000

department_name	total_salary
Database Support	20,000
Application Support	30,000
Software Support	15,000

Grouping Syntax

- Get each separate group to use an "aggregate" function over it

```
SELECT column_one,
       column_two
FROM table_name

GROUP BY column_one,
         column_two;
```

Grouping Columns

Aggregate Functions

SUM, MAX, MIN, AVG…

Aggregate Functions

- Used to operate over **one** or **more** groups performing **data analysis** on every one
  - SUM, MIN, MAX, AVG, etc.
- They usually **ignore** **NULL** values

department_id integer	min_salary numeric
1	1700
2	780
3	650
4	[null]

Aggregate Functions

- Use an "aggregate" function over each separate group

```
SELECT column_one,
       aggregate_function(column_two)
FROM table_name
GROUP BY column_one;
```

SUM

- **SUM** - sums the values in a column based on grouping criteria

employee	department_name	salary
Adam	Database Support	5,000
John	Database Support	15,000
Jane	Application Support	10,000
George	Application Support	15,000
Lila	Application Support	5,000
Fred	Software Support	15,000

department_name	total_salary
Database Support	20,000
Application Support	30,000
Software Support	15,000

SUM Syntax

- Problem & Solution: Sum Salaries per Department
- If all employees in a department have no salaries, **NULL** will be displayed

```
SELECT "department_id",
  SUM("salary") AS "total_salaries"
FROM "employees"
GROUP BY "department_id"
ORDER BY "department_id";
```

Grouping Column

MAX

- **MAX** - takes the maximum value in a column

employee	department_name	salary
Adam	Database Support	5,000
John	Database Support	15,000
Jane	Application Support	10,000
George	Application Support	20,000
Lila	Application Support	5,000
Fred	Software Support	15,000

department_name	max_salary
Database Support	15,000
Application Support	20,000
Software Support	15,000

MAX Syntax

- Problem & Solution: Maximum Salary per Department

```
SELECT "department_id",
   MAX("salary") AS "max_salary"
FROM "employees"
GROUP BY "department_id"
ORDER BY "department_id";
```

Grouping Column

MIN

- **MIN** - takes the minimum value in a column

employee	department_name	salary
Adam	Database Support	5,000
John	Database Support	15,000
Jane	Application Support	10,000
George	Application Support	15,000
Lila	Application Support	5,000
Fred	Software Support	15,000

department_name	min_salary
Database Support	5,000
Application Support	5,000
Software Support	15,000

MIN Syntax

- Problem & Solution: Minimum Salary per Department

```
SELECT "department_id",
  MIN("salary") AS "min_salary"
FROM "employees"
GROUP BY "department_id"
ORDER BY "department_id";
```

Grouping Column

AVG

- **AVG** - calculates the average value in a column

employee	department_name	salary
Adam	Database Support	5,000
John	Database Support	15,000
Jane	Application Support	10,000
George	Application Support	15,000
Lila	Application Support	5,000
Fred	Software Support	15,000

department_name	average_salary
Database Support	10,000
Application Support	10,000
Software Support	15,000

AVG Syntax

- Problem & Solution: Average Salary per Department

```
SELECT "department_id",
   AVG("salary") AS "avg_salary"
FROM "employees"
GROUP BY "department_id"
ORDER BY "department_id";
```

Grouping Column

Having

Using Predicates While Grouping

Having Clause

- The **HAVING** clause is used to filter data based on **aggregate** values
  - We cannot use it **without** grouping **before** that
- Any Aggregate functions in the "**HAVING**" clause and in the "**SELECT**" statement are executed one time only
- Unlike **HAVING**, the **WHERE** clause filters rows **before** the aggregation

Having Clause: Example

- Filter departments that have a **total** salary **less than** 25,000.

employee	department_name	salary	Total Salary
Adam	Database Support	5,000	20,000
John	Database Support	15,000	
Jane	Application Support	10,000	30,000
George	Application Support	15,000	
Lila	Application Support	5,000	
Fred	Software Support	15,000	15,000

Aggregated value

department_name	total_salary
Database Support	20,000
Software Support	15,000

HAVING Syntax

- Filter Total Salaries per Department, where total salary is less than 4200

```
SELECT "department_id",
       SUM("salary") AS "Total Salary"
FROM "employees"
GROUP BY "department_id"
HAVING SUM("salary") < 4200
ORDER BY "department_id";
```

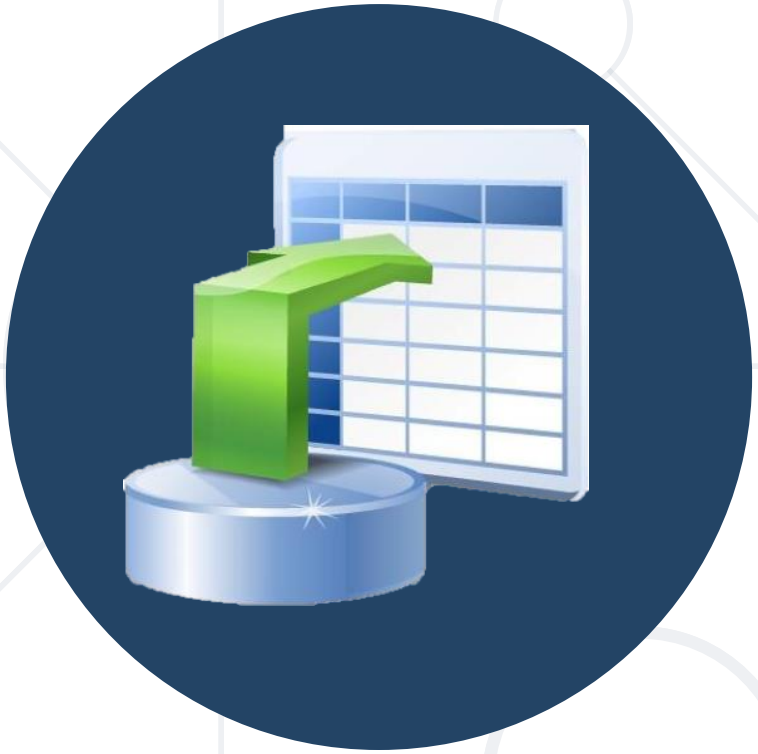
Aggregate Function

Grouping Column

Having Predicate

Live Demo

Aggregate Functions



Database Design

Fundamental Concepts

Steps in Database Design

Software University

1. Identification of the entities
2. Defining table columns
3. Defining primary keys
4. Modeling relationships
5. Defining constraints
6. Filling test data

Identification of Entities

- Entity tables represent objects from the real world
  - Most often they are nouns in the specification
  - For example:

```
We need to develop a system that stores
information about students, which are trained
in various courses. The courses are held in
different towns. When registering a new
student, the following information is entered:
name, faculty number, photo, and date.
```

  - Entities: **Student**, **Course**, **Town**

Identification of the Columns

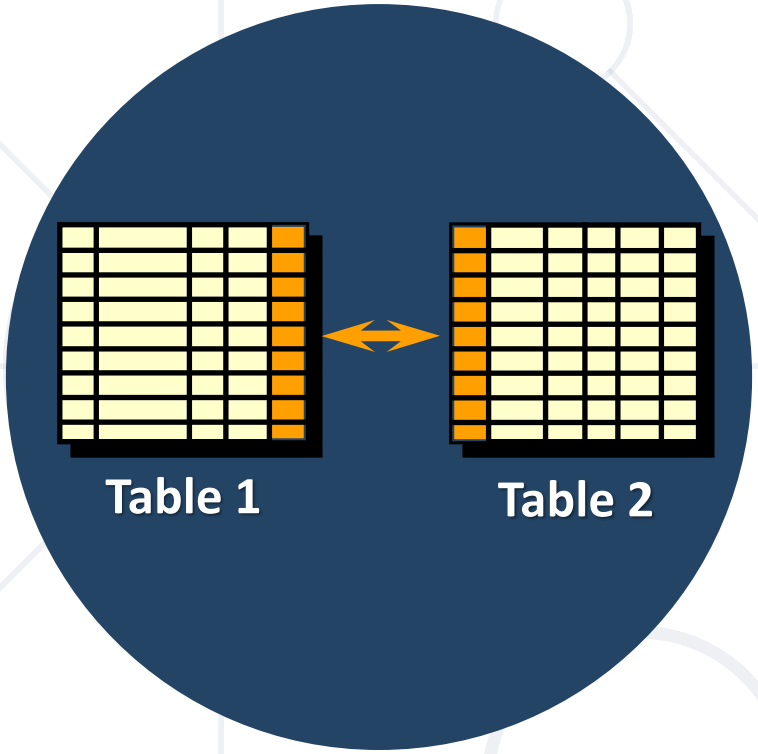
- Columns are the attributes of entities, defined in the specification's text, for example:

```
We need to develop a system that stores
information about students, which are trained
in various courses. The courses are held in
different towns. When registering a new
student, the following information is entered:
name, faculty number, photo, and date.
```

- Students have the following characteristics (attributes):
  - Name, faculty number, photo, date of enrolling, and a list of courses they visit

How to Choose a Primary Key?

- Always define an additional column for the primary key
  - Don't use an existing column (for example "name")
  - Can be an integer number
  - Must be declared as a `PRIMARY KEY`
  - Put the primary key in a first column
- Exceptions
  - Entities that have well-known ID, e.g., countries (BG, DE, US) and currencies (USD, EUR, BGN)



JOINs

Gathering Data from Multiple Tables

Data from Multiple Tables

- Sometimes you need data from **several tables**

Employees

EmployeeName	DepartmentID
Edward	3
John	NULL

Departments

DepartmentID	DepartmentName
3	Sales
4	Marketing
5	Purchasing

EmployeeName	DepartmentID	DepartmentName
Edward	3	Sales

Joins

- Table relations are useful when combined with JOINS
- With JOINS we can get data from two tables **simultaneously**
  - By pointing a "**join condition**"
  - Example:

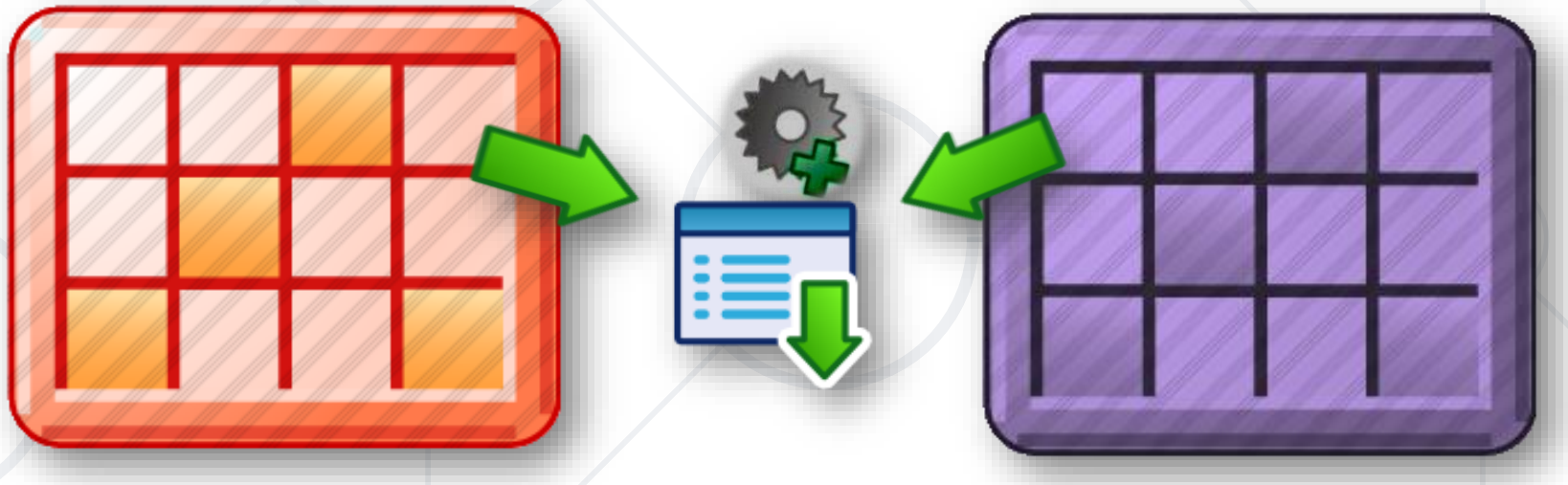
```
SELECT * FROM table_a
  JOIN table_b ON
    table_b.common_column = table_a.common_column
```

Select from Tables

Join Condition

Types of Joins

- **Inner** joins
- **Left**, **right** and **full outer** joins
- **Cross** joins



INNER vs. OUTER Joins

- **Inner** join
  - Join of two tables returning **only rows matching** the join **condition**
- **Left** (or **right**) **outer** join
  - Returns the results of the inner join as well as unmatched rows from the left (or right) table
- **Full outer** join
  - Returns the results of an **inner join** along with all **unmatched rows**

Inner Join

Employees

EmployeeID	DepartmentID
263	3
270	NULL

Departments

DepartmentID	DepartmentName
3	Sales
4	Marketing
5	Purchasing

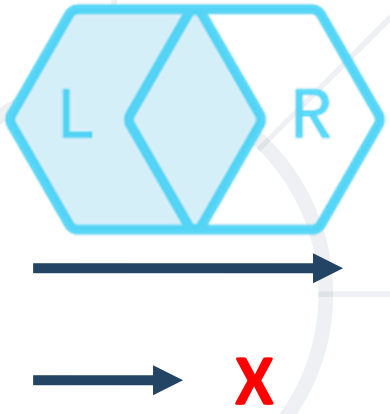
Result

EmployeeID	DepartmentID	DepartmentID	DepartmentName
263	3	3	Sales

Left Outer Join

Employees

EmployeeID	DepartmentID
263	3
270	NULL



Departments

DepartmentID	DepartmentName
3	Sales
4	Marketing
5	Purchasing

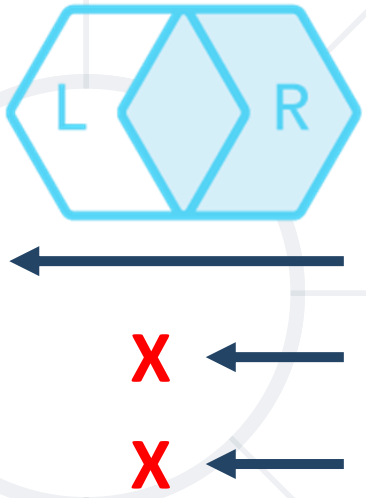
Result

EmployeeID	DepartmentID	DepartmentID	DepartmentName
263	3	3	Sales
270	NULL	NULL	NULL

Right Outer Join

Employees

EmployeeID	DepartmentID
263	3
270	NULL



Departments

DepartmentID	DepartmentName
3	Sales
4	Marketing
5	Purchasing

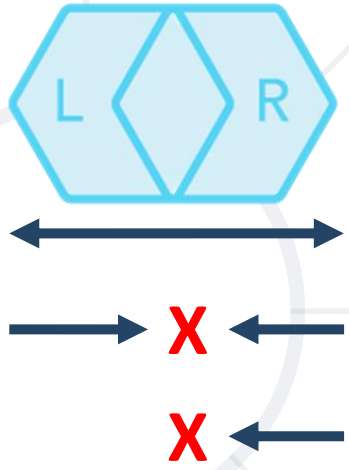
Result

EmployeeID	DepartmentID	DepartmentID	DepartmentName
263	3	3	Sales
NULL	NULL	4	Marketing
NULL	NULL	5	Purchasing

Full Join

Employees

EmployeeID	DepartmentID
263	3
270	NULL



Departments

DepartmentID	DepartmentName
3	Sales
4	Marketing
5	Purchasing

Result

EmployeeID	DepartmentID	DepartmentID	DepartmentName
263	3	3	Sales
270	NULL	NULL	NULL
NULL	NULL	4	Marketing
NULL	NULL	5	Purchasing

Live Demo

JOINs

Summary

- Data Management
- PostgreSQL
- Data Types
- Lexical structure in pgSQL
- Retrieving data
- Filtering
- Aggregate Functions
- JOINs

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg, about.softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity

Software University

FORUM

License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://about.softuni.bg/
- © Software University – https://softuni.bg